

Instructions: go through these questions, answering them elsewhere (there is not enough room to fit your answers). Work collaboratively, ask questions, but produce your own final work. See me with questions or concerns.

One big application of Machine Learning is that of predictive text, NLP, and LLMs. But how does this work? How do we even do math with words (or other categorical data)?

Towards that end, we will explore *embeddings*.

Essentially, embeddings are a numerical vectors assigned to our data. Here, those embeddings will be vector representing words.

The hope is that similar words are (or eventually become, though training) close to each other, giving a directionality to the meaning.

For this project, we will use a small vocabulary and some data (a corpus) to train on:

the cat sleeps  
the cat runs  
the dog runs  
the teacher teaches

The model's *vocabulary* is then: cat, dog, sleeps, runs, teacher, teaches

Enough info, let's do things!

Assign each of these words a random 2-dimensional nonzero vector (with components of magnitude  $\leq 1$ ):

cat =  $\langle 1, 1 \rangle$     dog =  $\langle 1, 0.5 \rangle$     runs =  $\langle -1, 1 \rangle$     sleeps =  $\langle 0.1, -0.1 \rangle$     teacher =  $\langle 1, -0.9 \rangle$     teaches =  $\langle -0.5, -0.5 \rangle$

1. Given our embeddings:
  - (a) Plot those vectors in the coordinate plane.
  - (b) Think about relations:
    - i. According to our choices, which words begin related?
    - ii. Which words should be related?
    - iii. What do I mean by related?
  - (c) What vector operation gives a sense of how much two vectors are similarly directed? What are the two ways to calculate it?  
[Talk to someone with Linear experience.]
  - (d) Make some informed predictions:
    - i. Why do we specify that the vectors are nonzero?
    - ii. Why might we want the components to be bounded by  $\pm 1$ ?

## 2. Training time:

Let's consider words pairs.

According to our corpus, an input of *cat* and *sleeps* should have an output of  $y = 1$

While *cat* and *teaches*, for example, should have an output of  $y = 0$

For *the cat sleeps*:

- (a) Passing forwards from the inputs, calculate

$$z = x_1x_2 + y_1y_2$$

- (b) Now calculate the sigmoid function  $\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$

- (c) Interpret your result.

[Recall what values the sigmoid function outputs. What would the takeaway be if  $\hat{y}$  is close to 1? to 0?]

- (d) So what does the model think about *cat sleeps*?

## 3. Ok, but we are supposed to learn:

$$L = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

- (a) Find the loss for *cat sleeps*.

- (b) Interpret your result.

## 4. Cool. But how should our embeddings change to reduce loss?

- (a) Gradient time! Show that

$$\frac{dL}{dz} = \hat{y} - y$$

[You may want to use the chain rule here:  $\frac{dL}{dz} = \frac{dL}{d\hat{y}} \cdot \frac{d\hat{y}}{dz}$ ]

When you step back and think about it, this is both nice and clean mathematically but also lovely conceptually: the gradient is simply the prediction minus ground truth.

- (b) For *cat sleeps*:

- i. Calculate the gradient for both components of *cat*.
- ii. Calculate the gradient for both components of *sleeps*.

- (c) Let's push along these values, updating our embeddings. How about a learning rate of 0.1 :

$$w_{\text{new}} = w - 0.1 \nabla L$$

- i. Update both components for *cat* and *sleeps*.
- ii. Looking at the results, how did these two embeddings qua vectors change (in relation to each other)?

This gradient descent process has us moving from the loss to the embedding, backwards through our process. This is called *backpropagation*.

5. Another one!

(a) Repeat the above process for *cat teaches*.

(b) What happens in this example?

6. Coding time and extrapolation. Let's repeat this process for word pairs many, many times.

(a) But before that: what do you expect will happen?

(b) I worry that our corpus might be too small. Let's expand it:

the cat sleeps

the dog sleeps

the bird sleeps

the cat runs

the dog runs

the bird flies

the student studies

the teacher teaches

the student learns

the dog chases the cat

the cat chases the bird

the teacher teaches the student

i. What is the vocabulary here?

ii. Share your code!

(c) Reflection:

i. Why do we require nonzero initial choices for our embeddings?

ii. Why are embeddings helpful?

iii. What is backpropagation doing, conceptually?

iv. Did we ever tell the model categories or meanings or anything about the words?

v. How or what is this model learning?

vi. If our embeddings were vectors with 100 components instead:

A. How would the above mathematical process change, if at all?

B. What would the model gain?